

CAB203 Graphs Project (Assessment 2)

Rory Blair: n10773398

1. Tournament Structure $\rightarrow$ gamesOK(games)

1.1). Mathematical Explanation

The objective of this project is to test whether different tournament structures will satisfy the requirements set out by the Committee to enable a tournament to commence and be completed. These requirements include that all players compete in the same number of games. Also, when a pair of players is created they can either play against each other or two other players that they both play against. The input is given in the format of a 'set' of 'tuples' where the 'set' is the proposed structure of a tournament, the tuples are the proposed games within the tournament and the values within the tuples are the players. (E.G. $\text{games} = \{\text{proposed games}\} \rightarrow \{(\text{player1}, \text{player2})\}$).

To satisfy the requirements of a tournament the use of two graph functions, from 'graphs.py' are used to help check for the satisfaction of the requirements. One of these functions is the 'degree' function which returns the number of neighbourhoods a given player has using 'V' and 'E' (Voloshin, 2009), which will be used to check if all players have the same number of games. The other function is 'N', which returns a 'set' that includes the neighbourhood of a player, or the adjacent vertices of the player given 'V' and 'E', which will check the other condition for a tournament (Voloshin, 2009). These conditions can be represented as quantified predicates as seen below:

$$\begin{aligned} \forall_{p1, p2, \dots, pn} \in V: \text{degree}(V, E, 1) &= \text{degree}(V, E, p2) = \dots = \text{degree}(V, E, pn) \\ \exists_{pp, pf} \in N(V, E, pi) \cap N(V, E, pj): &(pp \neq pi) \wedge (pp \neq pj) \wedge (pf \neq pi) \wedge (pf \neq pj) \end{aligned}$$

1.2). Mathematical Solution

To solve the problem, it is first required to find the vertices, 'V', as none are provided. The edges, 'E', are as the input, games. 'V' includes the players and 'E' is the edges/games. The next task is to find the 'degree' of each player and then the 'degree' of each player is checked to see if they are all the same. If the 'degree' of all players aren't the same, the tournament structure is considered not suitable. The next task would be to check if each player plays against another player or have at least two common opponents. To achieve this the program will iterate through 'V' checking if players aren't playing against each other and check that the number of common opponents is greater than two. If both of these conditions are true, then the structure of a tournament is considered suitable. Three test cases are seen in table 1:

	Test case 1	Test case 2	Test case 3
Games	$\{('p1', 'p2'), ('p2', 'p3')\}$	$\{('p1', 'p2'), ('p2', 'p3'), ('p1', 'p3')\}$	$\{('p1', 'p2'), ('p2', 'p3'), ('p3', 'p4'), ('p4', 'p5'), ('p5', 'p1')\}$
Degree of each player	False: p2 plays twice, and the others play once (output:[1,1,2])	True: They all play twice (output:[2,2,2])	True: They all play twice (output:[2,2,2,2,2])
Common Opponents	False before this section	True: They all have two opponents in common (output:3)	False: None of them have more than two other opponents in common (output:1)
Is Suitable	NO	YES	NO

Table 1: Tests for gamesOK(games)

1.3). Python Implementation

The important variables are 'E', 'V', 'S' & 'C'. To define variable 'E', as 'set' of tuples must be created. This variable is a symmetric version of the input 'games'. Once 'E' is created, 'V' can then be defined. This second variable includes all the players within 'E'. 'V' and 'E' are 'set' variables which will ensure that duplicates aren't included within 'V' and 'E'. Then the 'degree' of each player is found, using the 'degree' function from the `graphs.py` file, and put into a list, 'S'. Then this list 'S' is converted into a 'set'. Then, if the length of the 'set' version of 'S' is not equal to one, then every player isn't playing an equal number of games. This is because if all the numbers in the list 'S' are the same then the 'set' version of 'S' should just include the one number as no duplicates are included (McKague, CAB203 Lecture 4: Sets, 2024).

After testing the condition set out above, the program checks the other conditions, by iterating through, 'set' 'V'. The first condition is to check if the players do not play against each other. This is the first 'IF' condition within the nested 'FOR' loop. If the result is true, the program then looks at the common opponents for the two players, using the 'N' function from the `graphs.py` file, and put it in 'set', 'C'. If the length of 'set' 'C' is greater than two, then all conditions are true, and the tournament is suitable. If the result is false the tournament is not suitable.

2. Potential Referees → referees(games, refereecsvfilename)

2.1). Mathematical Explanation

The next requirement is that all games must have a referee assigned to them. The issue is that the referee can have a conflict of interest, creating a possible burden for the referee. The referee must declare their conflicts of interest, be assigned to one and must not be a player within the same game. The objective is to be able to find a solution where the referees are assigned to one game where the players within the game do not create a conflict of interest with that referee. The inputs given are the 'referees' csv file and the 'games'. This csv file includes the referee's conflicts of interest whilst the 'games' input will be the same input as the previous section (tournament structure).

To find a solution to this problem, the program will use a function from 'digraphs.py'. This 'maxMatching' function is being used to filter the possible assignment of referees to games considering the above mentioned requirements (McKague, CAB203 Lecture 8: Trees/Directed Graphs/Edge Weights, 2024). To do this the function returns a 'set' of undirected edges, given the bipartition 'A', 'B', and the original edges 'E'. These requirements for a referee can be seen as quantified predicates below:

$$\begin{aligned} \forall g \in G, \exists! r \in R: (g, r) \in \text{maxMatching}(A, B, E) \\ \forall r \in R, \{(g, r) \in \text{maxMatching}(A, B, E): g \in G\} \leq 1 \end{aligned}$$

2.2). Mathematical Solution

The solution starts with the creation of a referee dictionary, which includes the referee as the key and their conflicts as a 'set' stored as the 'item'. Next, the inputs for the 'maxMatching' function are initialised, 'A' is the games given to us, 'B' is a 'set' of the referees and 'E' is the edges between the games and the referees. Once the values have been initialised, the 'maxMatching' function is able to be used and the output will be a 'set' of undirected edges that include the best possible referee assignments for each game. Using the results from 'maxMatching' the final assignment for a referee can be computed by iterating through the results from 'maxMatching'. Within the iterations process, it checks if each iteration of the 'maxMatching' result is in the format of 'game, referee' where game is a tuple. If this condition is true, it adds the iteration to the final referee assignment, otherwise it keeps checking the other iterations/instances. After checking every iteration, it will check that the length of the final referee assignments isn't the same length as the games. If this statement is true, then that means that not every game has a referee, and none will be assigned. If it is false, the final referee assignment is the solution. Two test cases are seen in table.

	Test case 1	Test case 2
Games	{('p1', 'p2'), ('p3', 'p4'), ('p5', 'p6')}	{('p1', 'p2'), ('p3', 'p4'), ('p7', 'p6')}
Referee Dictionary	{ 'p3': {'p1', 'p7', 'p4', 'p9'}, 'p6': {'p7', 'p4', 'p8'}, 'p8': {'p2', 'p5', 'p7'} }	
'maxMatching'	{(('p5', 'p6'), 'p3'), ('p8', ('p3', 'p4')), ('p3', ('p5', 'p6')), (('p3', 'p4'), 'p8'), ('p6', ('p1', 'p2')), (('p1', 'p2'), 'p6')}	{('p8', ('p3', 'p4')), (('p3', 'p4'), 'p8'), (('p1', 'p2'), 'p6'), ('p6', ('p1', 'p2'))}
Results	{('p3', 'p4'): 'p8', ('p5', 'p6'): 'p3', ('p1', 'p2'): 'p6'}	{('p3', 'p4'): 'p8', ('p1', 'p2'): 'p6'}
All referees assigned	True: All referees are assigned as the length of results (3) is the same as the length of the games given (3)	False: not every game was assigned a referee as the result (2) were not the same length as the length of games (3)

Table 2: Tests for referees(games, refereecsvfilename)

2.3). Python Implementation

To start the referee dictionary 'R' is initialised so that the values from the csv file provided are able to be stored. A dictionary was used for 'R' as referee can be stored as the "key", whilst his conflicts as a 'set' could be stored as the "item". After extracting the referee and their conflicts of interest, the variables 'A', 'B' and 'E' are initialised, 'A' being the 'set' of games, 'B' being the 'set' referees and 'E' being the potential referee and game combinations. Putting these variables into the 'maxMatching' function produces a dictionary stored under the variable name 'M'. This dictionary includes the assignment of a referee to a game. Next, using the results from the above process, the program will iterate through each result and find the iterations where the format is 'game, referee' where game is the tuple. This part of the process is necessary as some of the results can be correct but instead of the format 'game, referee' they are 'referee, game'. The 'isinstance' function is used to resolve this. If the iteration is in the correct format, the program will then add the iteration to the results dictionary, 'RES'. In this dictionary the "key" is the tuple that is stored in game and the "item" is the referee. The final output is in this format as it was requested and because like 'sets', dictionary keys cannot be duplicates.

3. Assign Referee → gameGroups(assignedReferees)

3.1). Mathematical Explanation

The next part of the process is for the games to be scheduled. This is carried out in two phases. The initial task is to create games that can be played at the same time into game groups or game sessions. The requirements for these sessions are that people are not double-booked and a guarantee that each game is played once. The program needs to ensure that people are not double booked within a game session as a player or a referee. In addition, the program needs to ensure that each player plays at least one game per session. The committee is aiming for the minimum number of game sessions possible, meaning that every game group will be put in a single time slot. The inputs given are in the same format of the output from the previous section (Potential Referees), which is a dictionary of games where the keys are the games, and the items are the referee assigned to the game.

To find a possible solution to this problem, a function from 'graphs.py' will be used to organise the games into groups that can be played at the same time. The function that is being used is the 'minColoring' function, which returns the chromatic number, 'k', and a dictionary, C, which includes each value of V as the keys and the colour for each of them as the item (McKague, CAB203 Lecture 7: Graphs, 2024). The requirement for game groups can be seen as quantified predicate below:

$$\begin{aligned} \forall GG \forall p \in P \left(\sum_{g \in GG} 1_{p \in R(g)} \leq 1 \wedge \sum_{g \in GG} 1_{p \in P(g)} \leq 1 \right) \\ \forall g \in GG \exists! GG (g \in GG) \\ \forall GG \in GG \exists t \in T (GG = T(t)) \end{aligned}$$

3.2). Mathematical Solution

The solution starts with finding the values, 'V' and 'E' from the inputs given. In this case 'V' is the length of the game groups. 'E' will be transformed into a symmetric version of itself. After finding the values of 'V' and 'E', the 'minColouring' function is able to be utilised, to find the colouring of each game in vertices 'V'. Once the 'minColouring' has been computed, a list of game are able to be sorted into groups/sessions based on their computed colour. After assigning games to a certain session, the session is then added to a time slot within the game Groups, which is the final result. Two test cases are seen in table 3:

	Test case 1	Test case 2
assignedReferees	{('p1', 'p2'): 'p3', ('p4', 'p5'): 'p6'}	{('p1', 'p2'): 'p3', ('p4', 'p5'): 'p6', ('p3', 'p4'): 'p1', ('p6', 'p2'): 'p5', ('p1', 'p3'): 'p6'}
G	[('p1', 'p2'), ('p4', 'p5')]	[('p1', 'p2'), ('p4', 'p5'), ('p3', 'p4'), ('p6', 'p2'), ('p1', 'p3')]
V	range(0, 2)	range(0, 5)
E	[]	[(0, 3), (3, 0), (0, 4), (4, 0), (1, 2), (2, 1), (2, 4), (4, 2)]
C	{0: 0, 1: 0}	{0: 0, 1: 0, 2: 1, 3: 1, 4: 2}
GG = result	[{('p1', 'p2'), ('p4', 'p5')}]	[{('p4', 'p5'), ('p1', 'p2')}, {('p3', 'p4'), ('p6', 'p2')}, {('p1', 'p3')}]

Table 3: Tests for gameGroups(assignedReferees)

3.3). Python Implementation

The list of the games, 'named 'G'', comes from the input, 'assignedReferee'. This list receives the keys from the dictionary that are in fact the games. It then uses the length of 'G' to create 'V' which is a range of numbers from 0 to the length of 'G' minus one. The solution then finds the 'E' variable, which is a list that stores the game edges. Using both the 'V' and 'E' variables, the 'minColouring' function is able to be used to be able to assign certain games to particular groups, which is outputted as dictionary, 'C'. Then the program converts the values with 'C' to reasonable values because the values in 'C' are the numbers that correspond to a particular game. This newly converted version of 'C' is a list of 'sets', where each 'set' includes a game session and within the game session, there are tuples, which represent games.

4. Games Schedule → gameSchedule(assignedReferees, gameGroups)

4.1). Mathematical Explanation

The prior section (Assign Referees) only focused on assigning games to certain game groups based the fact a player could be in multiple games. The previous section had no regards for the fact that referees can also be players. The Committee now prefers that the person, who is a referee and a player within multiple game groups, should participate in all their games, as a player, before being allowed to referee. The program will need to determine if a particular group of games is able to be rearranged so as to satisfy this additional requirement. The inputs given are the ‘assignedReferees’, the same input as the prior section, and the ‘gameGroups’, which is also the output of the prior section. The ‘assignedReferees’ is a dictionary where the games/tuples are the ‘key’ values, and the referee of that game is the ‘item’ value. The ‘gameGroups’ is a list of ‘sets’ where each ‘set’ is a single session and within each ‘set’ are tuples that are the games that are going to be played in that games session.

To find a solution to this problem a function from ‘digraphs.py’ will be used to help rearrange the order of ‘gameGroups’ so that a person that is both a referees and a players are allocated to a game as a player before they are allocated to a game as a referee. The function being used is ‘topOrdering’, which returns a list that has the ‘gameGroups’ list reorganised in topological order (McKague, CAB203 Lecture 8: Trees/Directed Graphs/Edge Weights, 2024). The requirement for scheduling games can be seen as quantified predicate below:

$$\begin{aligned}
 &P(x) \wedge G(x, y) \rightarrow \neg R(x) \\
 &\forall y \exists x \left((P(x) \wedge G(x, y) \wedge \neg R(x)) \rightarrow \exists z (R(z) \wedge G(x, y)) \right) \\
 &\forall x \forall y \left((R(x) \wedge G(x, y)) \rightarrow \forall z (P(z) \wedge G(x, y) \wedge \neg R(z) \rightarrow B(z, x)) \right)
 \end{aligned}$$

4.2). Mathematical Solution

To find the solution, the first variable that have to be initialised are the ‘R’, ‘V’ and ‘E’ variables. The ‘R’ variable is a ‘set’ that includes all the players that are referees. The variable ‘V’ is a ‘set’ that ranges from 0 to the length of variable ‘gameGroups’. The variable ‘E’ is a ‘set’ of edges that relates to the variable ‘V’. Once these variables have been created, the ‘topOrdering’ function is able to be used to find the best possible schedule. Once the ‘topOrdering’ function has been used, the output, which should be a list of numbers in a particular order, can be used to rearrange the ‘gameGroups’ to satisfy the requirement given for a game schedule. Two test cases are seen in table 4:

	Test case 1	Test case 2
assignedReferees	{('p1', 'p2'): 'p3', ('p3', 'p2'): 'p4'}	{('p5', 'p6'): 'p7', ('p8', 'p5'): 'p9', ('p10', 'p11'): 'p8', ('p6', 'p12'): 'p5', ('p12', 'p13'): 'p6', ('p13', 'p10'): 'p11'}
gameGroups	[{('p1', 'p2')}, {('p3', 'p2')}]	[{('p12', 'p13')}, {('p13', 'p10'), ('p6', 'p12')}, {('p5', 'p6'), ('p10', 'p11')}, {('p8', 'p5')}]
V	{0, 1}	{0, 1, 2, 3}
E	set()	set()
O	[0, 1]	[0, 1, 2, 3]
GG = result	[{('p3', 'p2')}, {('p1', 'p2')}]	[{('p8', 'p5')}, {('p10', 'p11'), ('p5', 'p6')}, {('p6', 'p12'), ('p13', 'p10')}, {('p12', 'p13')}]

Table 4: Tests for gameSchedule(assignedReferees, gameGroups)

4.3). Python Implementation

The variable 'R' is a 'set' as it gets the referees allocated in no particular order. The variable 'V' and 'E' are also 'sets' as the inputs for the 'topOrdering' function require the two inputs to be formatted as 'sets'. The variable 'V' includes the sessions as numbers from zero to the length of 'gameGroups' minus one. The 'O' variable is the output of the 'topOrdering' process, which is a list meaning it is easier to iterate through and get specific values. The 'RG' variable is a list that is the rearranged version of 'gameGroups', which allow for the groups to be ordered into a specific order. The 'ROG' variable is a list only made to iterate through and add values to the 'RG' variable.

5. Player Ranking → scores(p, s, c, games)

5.1). Mathematical Explanation

Once all matches have been played, determining the winners of the tournament is required. The tournament uses a point-based system to determine the overall winners of the tournament. Given that all players don't play against each other, the number of wins isn't enough to determine a single winner, as someone could be more skilled and still not win, based on the fact that they didn't play as many games as others. To address this issue, the Committee wishes for players to earn points not only from their own wins but also from wins by opponents they have defeated. The scores for each player are calculated in six steps.

Step one is that the players must write down the other players they defeated also called primary wins. Above each opponent the player then writes the list of all opponents that their primary wins have defeated. They then draw a line from a primary win to a secondary win as a crosscheck to ensure the written results are correct. The second step is that the referee places a number of tokens, 'p', on the players primary wins. The third step is that the referee places a 's', on the players secondary wins. The fourth step is the player can then choose a line to follow with each token and move along from secondary to primary wins. The fifth step is that while doing step 4, the primary wins of each player can accommodate a limited number of tokens, 'c'. in saying this, not all secondary wins cannot be transferred to primary wins. The final step is where the player tallies his primary and secondary wins to determine their total score for the tournament.

To find a solution to this problem no functions from either the 'digraphs.py' or 'graphs.py' files need to be used. The inputs given to us are 'p', 's', 'c' and games. The input variable 'p' represents the primary win tokens given to each player. The input variable 's' represent the secondary win tokens given to each player. The input variable 'c' is the number that puts a capacity on how many primary win tokens players can have. The input variable 'games' is a 'set' of tuples, like the previous sections, but are actually the game results as well. Players to the left side of the tuple are the winners and the right side are the losers. The requirement for calculating scores can be seen as quantified predicate below:

$$\begin{aligned}
 & \text{Winner}(x) \wedge \text{Loser}(y) \rightarrow \exists z (\text{Winner}(x) \wedge \text{Loser}(z) \wedge \text{Winner}(z) \wedge \text{Loser}(y)) \\
 & \text{PrimaryWin}(x, z) \wedge \text{PrimaryWin}(z, y) \rightarrow \exists w (\text{Winner}(x) \wedge \text{Loser}(y) \wedge \text{Winner}(w) \wedge \text{Loser}(w)) \\
 & \text{PrimaryWin}(x, y) \rightarrow \exists m (\text{Winner}(x) \wedge \text{Loser}(y) \wedge m \leq p) \\
 & \text{SecondaryWin}(x, y) \rightarrow \exists m, n (\text{Winner}(x) \wedge \text{Loser}(y) \wedge m + n \leq s) \\
 & \quad \exists n (\text{Winner}(x) \wedge \text{Loser}(x) \wedge n \leq c) \\
 & \quad \exists n (\text{Winner}(x) \wedge \text{Loser}(x))
 \end{aligned}$$

5.2). Mathematical Solution

The solution to this problem starts by finding the primary wins and the secondary wins from the variable input, 'games'. As said above, winners are the left side, losers are the right side. Once the primary and secondary wins have been found, the next task is to get all the players into one single 'set'. Then the program iterates through the 'set' of all the players checking certain conditions. The first condition it checks is if the player is included in the primary wins, which if is false the total score is zero but if true the program continues to check the second condition. The second condition checks if the player is in the secondary wins. If true, the player has a primary score but no secondary score, which is calculated. If false, the player has a primary and secondary score that are calculated. After these conditions have been tested the program adds the primary and secondary score of each player and puts them into their respective positions in the dictionary. A test case is seen in table 5:

	Test case 1
P	3
S	2
c	4
games	{('Alice', 'Bob'), ('Alice', 'Dave'), ('Bob', 'Charlie'), ('Dave', 'Charlie')}
Primary Wins	{'Alice': {'Dave', 'Bob'}, 'Bob': {'Charlie'}, 'Dave': {'Charlie'}}
Secondary Wins	{'Bob': {'Alice'}, 'Dave': {'Alice'}, 'Charlie': {'Dave', 'Bob'}}
Primary and Secondary score	Primary Score: Dave 3 Secondary Score: Dave 0 Primary Score: Bob 3 Secondary Score: Bob 0 Primary Score: Alice 6 Secondary Score: Alice 2
Scores = results	{'Dave': 3, 'Bob': 3, 'Alice': 8, 'Charlie': 0}

Table 5: Test for scores(p, s, c, games)

5.3). Python Implementation

The variables 'PW', 'SW', 'P', 'S', 'ps' and 'ss' are created throughout the program. The variable 'PW' is a dictionary that includes the players as the 'key' values and a 'set' of player they have defeated as the 'item' value. The variable 'SW' is a dictionary that include the players as the 'key' values and a 'set' of player they have lost to as the 'item' value. The dictionary data structure was used for the 'PW' and 'SW' variables because it was the only way to store a player and their respective wins and loses in an readable and organised format. The variable 'P' is a 'set' that includes all the players within the games. The variable 'S' is a dictionary where the score for each player is stored after the final scores have been calculated. This variable is also a dictionary because it simplifies the readability of the results. The variable 'ps' is the number of other players the player has defeated, which is then multiplied by the value 'p'. The variable 'ss' is the number of players the player has defeated, which is not multiplied by 's' as they only get one point for each secondary win, meaning the multiplier would just be one.

6. References

- McKague, M. (2024, May 15). *CAB203 Lecture 4: Sets*. Retrieved from QUT:
https://canvas.qut.edu.au/courses/16665/pages/4-dot-2-lecture-material?module_item_id=1551958
- McKague, M. (2024, May 15). *CAB203 Lecture 7: Graphs*. Retrieved from QUT:
https://canvas.qut.edu.au/courses/16665/pages/7-dot-2-lecture-material?module_item_id=1551977
- McKague, M. (2024, May 15). *CAB203 Lecture 8: Trees/Directed Graphs/Edge Weights*. Retrieved from QUT:
https://canvas.qut.edu.au/courses/16665/pages/8-dot-2-lecture-material?module_item_id=1551983
- Voloshin, V. (2009). *Introduction to Graph Theory*. Nova Science Publishers, Incorporated.